

UNIVERSIDAD DE GUANAJUATO



Programación en Ingenierías

Funciones

En la práctica es importante dividir un programa en unidades razonables e independientes, por ello, la mayoría de los lenguajes de programación, incluido C, proporcionan una forma de dividir un programa en segmentos, cada uno de los cuales puede escribirse de forma más o menos independiente de los demás.

En C, estos segmentos se denominan funciones. El código de programa en el cuerpo de una función está aislado del de otras funciones. Una función tendrá una interfaz específica con el mundo exterior en términos de cómo se le transfiere la información y cómo se transmiten los resultados generados por la función. Esta interfaz se especifica en la primera línea de la función, donde aparece el nombre de la función.

En C, el concepto de función se puede explicar de varias formas. Por ejemplo:

- Una función es un subprograma. Te permite romper una gran tarea de computación en otras más pequeñas.
- Una función es un fragmento de código delimitado por llaves que realiza alguna tarea bien definida y también devuelve un valor.
- Una función es un componente básico de un programa. Es útil para hacer código existente reutilizable.
- Una función también se trata como un tipo derivado en C.
- Una función es una forma de ampliar el repertorio del lenguaje C

Todas las funciones deben codificarse. Codificar una función significa escribir el instrucciones de programación para esa función.

Las funciones en C se pueden clasificar en tres categorías:

- La función *main* (): esta función es necesaria en un programa en C. Habrá una (y solo una) función *main* () en cada programa en C. Cuando el sistema operativo ejecuta un programa en C, en realidad ejecuta la función *main* (). La función *main* () llama a la biblioteca funciones y funciones definidas por el usuario, según los requisitos. Se requiere que el programador codifique la función *main* ().
- Funciones de biblioteca: también se denominan definidas por el sistema o integradas. funciones. Por ejemplo, `printf` () y `scanf` () son bibliotecas funciones. Las funciones de la biblioteca son opcionales en un programa en C. Pero Es prácticamente imposible crear un programa útil que carezca de funciones de biblioteca. No es necesario que un programador codifique funciones de biblioteca. Los desarrolladores del compilador codifican las funciones de la biblioteca, compílelos y colóquelos en bibliotecas para su uso.
- Funciones definidas por el usuario: las funciones definidas por el usuario son opcionales en un programa C. Una función definida por el usuario llama a otra función definida por el usuario funciones o funciones de biblioteca, según los requisitos. La Se requiere que el programador codifique las funciones definidas por el usuario.

El último punto a tener en cuenta, antes de entrar en detalles sobre la creación de funciones, es que el cuerpo de cada función es un bloque (que puede contener otros bloques, por supuesto).

Como resultado, las variables automáticas que declara dentro de una función son local para la función y no existe en ningún otro lugar. Por lo tanto, las variables declaradas dentro de una función son bastante independientes de los declarados en otra función o en un bloque anidado.

No hay nada que le impida usar el mismo nombre para variables en diferentes funciones; permanecerán bastante separados. De hecho, esto es una ventaja. Eso sería muy difícil seguir creando nombres distintos para todas las variables en un programa grande. Es útil para poder usar el mismo nombre, como contador en diferentes funciones. Sigue siendo una buena idea evitar cualquier superposición engañosa de nombres de variables en sus diversas funciones y, por supuesto, debe intentar usar nombres que son importantes para que sus programas sean fáciles de seguir.

Declaración de una función.

Cuando crea una función, especifica el encabezado de la función como la primera línea de la declaración de la función, seguida de las instrucciones que serán ejecutadas en la función encerrada entre llaves. El bloque de código entre llaves que sigue del encabezado de la función se denomina cuerpo de la función.

- *El encabezado de la función* define el nombre de la función, los parámetros de la función (que especifican el número y los tipos de valores que se pasan a la función cuando se llama) y el tipo por el valor que devuelve la función.
- El *cuerpo de la función* contiene las instrucciones que se ejecutan cuando se llama a la función, y estos tienen acceso a los valores que se pasan como argumentos a la función.

```
Tipo_de_dato_de_retorno identificador( Parámetros de entrada - separados por coma )  
{  
  // Instrucciones ...  
}
```

Las declaraciones en el cuerpo de la función pueden estar ausentes, pero las llaves deben estar presentes. Si no hay declaraciones en el cuerpo de una función, el tipo de retorno debe ser nulo y la función no tendrá ningún efecto. En este caso es utilizado el tipo void que significa "ausencia de cualquier tipo", y aquí significa que la función no devuelve un valor.

Una función que tiene declaraciones en el cuerpo de la función pero no devuelve un valor también debe tener el tipo de retorno especificado como void.

Por el contrario, para una función que no tiene un tipo de retorno nulo, cada declaración de retorno en el cuerpo de la función debe devolver un valor del tipo de retorno especificado.

Definir una función con un cuerpo casi libre de contenido suele ser útil durante la fase de prueba de un programa complicado, por ejemplo, una función definida de modo que solo contiene una declaración de retorno, quizás devolviendo un valor predeterminado. Esto le permite ejecutar el programa con solo las funciones seleccionadas que realmente están haciendo algo; luego puede agregar el detalle de los cuerpos de función paso a paso, probando en cada etapa, hasta que todo se implemente y completamente probado.

El término parámetro se refiere a un marcador de posición en una definición de función que especifica el tipo de valor que debe pasarse a la función cuando se llama. El valor pasado a una función correspondiente a un parámetro se denomina argumento. Un parámetro de función consta del tipo seguido del nombre del parámetro que se utiliza dentro del cuerpo de la función para hacer referencia al valor de argumento correspondiente que se transfiere.

FUNCIONES / Declaración y prototipado de funciones



UNIVERSIDAD DE
GUANAJUATO

```
// Ejemplo001.cpp: Forma de declaración por prototipado
//

#include <stdio.h>

#define N 500000

float tan(float x, int n);
float sin(float x, int n);
float cos(float x, int n);

int main(int argc, char *argv[])
{
    float x;
    int n;
    printf("Ingrese el valor de x: ");
    scanf("%f", &x);
    do {
        printf("Ingrese el numero de terminos: ");
        scanf("%d", &n);
    } while (n < 1 || n > N);
    printf("tan(%f) = %f\n", x, tan(x, n));
    getchar();
    getchar();
    return 0;
}

float tan(float x, int n)
{
    return sin(x, n) / cos(x, n);
}

float sin(float x, int n)
{
    float fct, sx;
    int i, s;
    for (i = 1, fct = x, s = 1, sx = 0; i < n; i++)
    {
        sx += (s*fct);
        s *= (-1);
        fct *= ((x/(2*i))*(x/(2*i+1)));
    }
    return sx;
}

float cos(float x, int n)
{
    static int s = 1, i = 1;
    static float fct = 1;
    s *= (-1);
    fct *= ((x/i)*(x/(i+1)));
    i += 2;
    if (n)
        return s*fct + cos(x, n - 1);
    else
        return 1;
}
```

Comentarios

Librerías

Pre-procesamiento

Prototipos de funciones

Función main

Funciones

```
// Ejemplo002.cpp: Forma de declaración por declaración
//

#include <stdio.h>

#define N 500000

float sin(float x, int n)
{
    float fct, sx;
    int i, s;
    for (i = 1, fct = x, s = 1, sx = 0; i < n; i++)
    {
        sx += (s*fct);
        s *= (-1);
        fct *= ((x/(2*i))*(x/(2*i+1)));
    }
    return sx;
}

float cos(float x, int n)
{
    static int s = 1, i = 1;
    static float fct = 1;
    s *= (-1);
    fct *= ((x/i)*(x/(i+1)));
    i += 2;
    if (n)
        return s*fct + cos(x, n - 1);
    else
        return 1;
}

float tan(float x, int n)
{
    return sin(x, n) / cos(x, n);
}

int main(int argc, char *argv[])
{
    float x;
    int n;
    printf("Ingrese el valor de x: ");
    scanf("%f", &x);
    do {
        printf("Ingrese el numero de terminos: ");
        scanf("%d", &n);
    } while (n < 1 || n > N);
    printf("tan(%f) = %f\n", x, tan(x, n));
    getchar();
    getchar();
    return 0;
}
```

Comentarios

Librerías

Pre-procesamiento

Funciones

Función main

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

$$x^n = x \times x \times x \times \dots \times x$$

$$n! = 1 \times 2 \times 3 \times \dots \times n$$

```
float potencia(float x, int n)
```

```
{  
}
```

```
float factorial(int n)
```

```
{  
}
```

```
float exp(float x, int n)
```

```
{  
}
```

```
int main()
```

```
{  
}
```

```
float exp(float x, int n);
```

```
float factorial(int n);
```

```
float potencia(float x, int n);
```

```
int main()
```

```
{  
}
```

```
float potencia(float x, int n)
```

```
{  
}
```

```
float factorial(int n)
```

```
{  
}
```

```
float exp(float x, int n)
```

```
{  
}
```

Una función *recursiva* es aquella que se define en términos de sí misma.

El ejemplo más común es la función factorial. El factorial de un número entero no negativo, n (¡escrito como $n!$), se define de la siguiente manera:

$$n! = 1 \times 2 \times 3 \times 4 \times \cdots \times n$$

Por ejemplo:

$$\begin{array}{lll} 5! = 1 \times 2 \times 3 \times 4 \times 5 & & \\ 4! = 1 \times 2 \times 3 \times 4 & 5! = 4! \times 5 & \text{fact}(5) = \text{fact}(4) * 5 \\ 3! = 1 \times 2 \times 3 & 4! = 3! \times 4 & \text{fact}(4) = \text{fact}(3) * 4 \\ 2! = 1 \times 2 & 3! = 2! \times 3 & \text{fact}(3) = \text{fact}(2) * 3 \\ 1! = 1 & 2! = 1! \times 1 & \text{fact}(2) = \text{fact}(1) * 2 \\ 0! = 1 & 1! = 0! \times 1 & \text{fact}(1) = \text{fact}(0) * 1 \end{array}$$

```
int fact(int n)
{
    if(n>0)
        return fact(n-1)*n;
    else
        return 1;
}
```

fact(5)	return fact(4)*5	return 24*5=120
fact(4)	return fact(3)*4	return 6*4=24
fact(3)	return fact(2)*3	return 2*3=6
fact(2)	return fact(1)*2	return 1*2=2
fact(1)	return fact(0)*1	return 1*1=1
fact(0)	return 1	

$$x^6 = x^5 \times x$$

$$x^5 = x^4 \times x$$

$$x^4 = x^3 \times x$$

$$x^3 = x^2 \times x$$

$$x^2 = x^1 \times x$$

$$x^1 = x^0 \times x$$

$$x^0 = 1$$

$$x^{-1} = x^0 \times \frac{1}{x}$$

$$x^{-2} = x^{-1} \times \frac{1}{x}$$

$$x^{-3} = x^{-2} \times \frac{1}{x}$$

$$x^{-4} = x^{-3} \times \frac{1}{x}$$

$$x^{-5} = x^{-4} \times \frac{1}{x}$$

$$x^{-6} = x^{-5} \times \frac{1}{x}$$

$$\text{pot}(x,6)=\text{pot}(x,5)*x$$

$$\text{pot}(x,5)=\text{pot}(x,4)*x$$

$$\text{pot}(x,4)=\text{pot}(x,3)*x$$

$$\text{pot}(x,3)=\text{pot}(x,2)*x$$

$$\text{pot}(x,2)=\text{pot}(x,1)*x$$

$$\text{pot}(x,1)=\text{pot}(x,0)*x$$

$$\text{pot}(x,0)=1$$

$$\text{pot}(x,-1)=\text{pot}(x,0)*(1/x)$$

$$\text{pot}(x,-2)=\text{pot}(x,-1)*(1/x)$$

$$\text{pot}(x,-3)=\text{pot}(x,-2)*(1/x)$$

$$\text{pot}(x,-4)=\text{pot}(x,-3)*(1/x)$$

$$\text{pot}(x,-5)=\text{pot}(x,-4)*(1/x)$$

$$\text{pot}(x,-6)=\text{pot}(x,-5)*(1/x)$$

```
#include <stdio.h>
```

```
float pot(float x, int n);
```

```
int main(void) {
    float x, xn;
    int n;
    printf("Ingresa el valor de x: ");
    scanf("%f", &x);
    printf("Ingresa el valor de n: ");
    scanf("%d", &n);
    xn = pot(x, n);
    printf("%f^%d = %f\n", x, n, xn);
    return 0;
}
```

```
float pot(float x, int n)
{
    if(n>0)
        return pot(x,n-1)*x;
    else if(n<0)
        return pot(x,n+1)*(1/x);
    else
        return 1;
}
```

Alcance de variables.

De forma predeterminada, las variables tienen un alcance local, su uso se limita hasta el final del bloque en el que fueron definidas, y su memoria en la pila queda libre para que otra función la use. Se denominan variables automáticas porque se crean automáticamente en el punto en el que se declaran y se destruyen automáticamente. cuando la ejecución del programa sale del bloque en el que fueron declarados.

Este es un proceso muy eficiente porque los datos que contienen memoria en una función solo se retienen mientras esté ejecutando declaraciones dentro de la función en la que se declara la variable. Sin embargo, existen algunas circunstancias en las que es posible que desee retener información de una llamada a la función a la siguiente.

Es posible que desee mantener un recuento de algo dentro de una función, como el número de veces que se ha llamado a la función o el número de líneas de salida que se han escrito.

Con las variables automáticas, no tiene forma de hacer esto. Sin embargo, C le proporciona una forma de hacer esto con variables *static*. Una variable *static* tiene al mismo alcance que una variable global, pero la variable *static* solo es visible por la función donde fue declarada.

Alcance de variables.

Por ejemplo, usemos una variable automática y otra *static* para almacenar los valores un contador, realizaremos el llamado entre ambas funciones para verificar el funcionamiento. Es decir:

```
#include <stdio.h>           // Función test1 con variable automática
                               void test1(void)
                               {
                               int count = 0;
                               printf("test1 count = %d\n", ++count );
                               }
                               test1 count = 1
                               test2 count = 1
                               test1 count = 1
                               test2 count = 2
                               test1 count = 1
                               test2 count = 3
                               test1 count = 1
                               test2 count = 4
                               test1 count = 1
                               test2 count = 5

void test1(void);
void test2(void);
int main(void)
{
for(int i = 0 ; i < 5 ; ++i)
{
// Función test 2 con variable static
void test2(void)
{
static int count = 0;
printf("test2 count = %d\n", ++count );
}
test1();
test2();
}
return 0;
}
```

Alcance de variables.

Por ejemplo, usemos una variable *static* para almacenar los valores entre llamados, que nos permita obtener la potencia de x. Es decir:

$$y = x^n$$

```
#include <stdio.h>
```

```
float pot(float x, int n)
```

```
{
```

```
    static float px=1;
```

```
    if(n!=0)
```

```
    {
```

```
        px*=x;
```

```
        pot(x,n-1);
```

```
    }
```

```
    return px;
```

```
}
```

```
pot(1.5,6);
```

```
    px=1;
```

```
    px*=x->px=1.5
```

```
    pot(1.5,5)
```

```
pot(1.5,5)
```

```
    px*=x->px=2.25
```

```
    pot(1.5,4)
```

```
pot(1.5,4)
```

```
    px*=x->px=3.375
```

```
    pot(1.5,3)
```

```
pot(1.5,3)
```

```
    px*=x->px= 5.0625
```

```
    pot(1.5,2)
```

```
pot(1.5,2);
```

```
    px*=x->px=7.59375
```

```
    pot(1.5,1)
```

```
pot(1.5,1)
```

```
    px*=x->px=11.390625
```

```
    pot(1.5,0)
```

```
pot(1.5,0)
```

```
    return 11.390625;
```

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

$$1 = \frac{x^0}{0!} = 1$$

$$\frac{x^1}{1!} = 1 \times \frac{x}{1}$$

$$\frac{x^2}{2!} = \frac{x^1}{1!} \times \frac{x}{2}$$

$$\frac{x^3}{3!} = \frac{x^2}{2!} \times \frac{x}{3}$$

$$\frac{x^4}{4!} = \frac{x^3}{3!} \times \frac{x}{4}$$

$$\frac{x^n}{n!} = \frac{x^{n-1}}{(n-1)!} \times \frac{x}{n}$$

```
#include <stdio.h>
```

```
void fexp(float x, int n);
```

```
int main(void) {
    float x;
    int n;
    printf("Ingrese el valor de x: ");
    scanf("%f", &x);
    do{
        printf("Ingrese el numero de terminos: ");
        scanf("%d", &n);
    }while(n<1);
    fexp(x,n);
    return 0;
}
```

```
void fexp(float x, int n)
{
    static float fct = 1;
    static int i = 1;
    printf("%f\n", fct);
    fct*=(x/i++);
    if(i<=n)
        fexp(x,n);
}
```

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

$$ex = 1$$

$$ex + x = 1 + x = ex$$

$$ex + \frac{x^2}{2!} = 1 + x + \frac{x^2}{2!} = ex$$

$$ex + \frac{x^3}{3!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} = ex$$

$$ex + \frac{x^4}{4!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} = ex$$

```
#include <stdio.h>
```

```
float fexp(float x, int n);
```

```
int main(void) {
    float x, ex;
    int n;
    printf("Ingrese el valor de x: ");
    scanf("%f", &x);
    do{
        printf("Ingrese el numero de terminos: ");
        scanf("%d", &n);
    }while(n<1);
    ex=fexp(x,n);
    printf("e(%f) = %f\n", x, ex);
    return 0;
}
```

```
float fexp(float x, int n)
{
    static float fct = 1;
    float fct2;
    static int i = 1;
    fct2 = fct;
    fct*=(x/i++);
    if(i<n)
        return fct2+fexp(x,n);
    else
        return fct2;
}
```

Odd Divisors

Time limit: *1000 ms*

Memory limit: *128 MB*

You are given an interval of integers $[A,B]$. For each number in this interval compute its greatest odd divisor. Output the sum of these divisors.

Standard input

The first line contains an integer T representing the number of test cases that will follow. Each test case consists of one line containing two integer values A and B .

Standard output

The output should contain the answer for each test case on a different line. Each answer consists of a single integer value.

Constraints and notes

$$1 \leq T \leq 10^5$$

$$1 \leq A \leq B \leq 10^9$$

Odd Divisors

3
1 3
1 4
3 9

5
6
29

$[1\ 3] = 1, 2, 3$

$1 = 1$

$2 = 1, 2 \quad 1+1+3=5$

$3 = 1, 3$

$[3\ 9] = 3, 4, 5, 6, 7, 8, 9$

$3=3$

$4=4,2,1$

$5=5$

$6=6,3$

$7=7$

$8=8,4,2,1$

$9=9$

$3+1+5+3+7+1+9=29$

$[1\ 2] = 1+1=2$

$[1\ 9] = 1+1+3+1+5+3+7+1+9=31$

$[1\ 9]-[1\ 2] = 31-2 = 29$

Odd Divisors

$$9 + 9\%2 = 10$$

$$\left\lfloor \frac{4}{2} \right\rfloor = 2$$

$$x = \begin{bmatrix} 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{bmatrix} \rightarrow \begin{bmatrix} 3 \\ 1 \\ 5 \\ 3 \\ 7 \\ 1 \\ 9 \end{bmatrix} = 29$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \\ 3 \\ 1 \\ 5 \\ 3 \\ 7 \\ 1 \\ 9 \\ 5 \end{bmatrix} = \frac{10}{2} \times \frac{10}{2} = 5 \times 5 = 25$$

$$2 + 2\%2 = 2$$

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{2}{2} \times \frac{2}{2} = 1 \times 1 = 1$$

$$x_i = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 2$$

$$\left\lfloor \frac{2}{2} \right\rfloor = 1$$

$$x_s = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \\ 3 \\ 1 \\ 5 \\ 3 \\ 7 \\ 1 \\ 9 \end{bmatrix} = 31$$

$$[1] \rightarrow [1] = 1$$

$$25 + 4 + 1 + 1 = 31$$

$$\left\lfloor \frac{9}{2} \right\rfloor = 4$$

$$4 + 4\%2 = 4$$

$$x_s - x_i = 31 - 2 = 29$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \\ 3 \end{bmatrix} = \frac{4}{2} \times \frac{4}{2} = 2 \times 2 = 4$$

Odd Divisors

```
#include <stdio.h>
#include <math.h>

long int sumOD4(long int A, long int B);
long int capturar(void);
long int sDI2(long int x);

int main(void)
{
    long int T, A, B, i;
    T=capturar();
    for(i=0; i<T; i++)
    {
        A=capturar();
        B=capturar();
        printf("%ld\n", sumOD4(A, B));
    }
    return 0;
}

long int sumOD4(long int A, long int B)
{
    return sDI2(B)-sDI2(A-1);
}
```

```
long int sDI2(long int x)
{
    long int y;
    long int m;
    if(x>1)
    {
        m = x+(x&1);
        y = m*m>>2;
        return y+sDI2(x>>1);
    }
    else if(x==1)
        return 1;
    else
        return 0;
}

long int capturar(void)
{
    long int X=0;
    char c;
    c=getchar();
    while(c<=32)
        c=getchar();
    while('0'<=c&& c<='9')
    {
        X=(10*X+(c-'0'));
        c=getchar();
    }
    return X;
}
```

Odd Divisors

```
#include <stdio.h>
```

```
long int DivImp(long int n);  
long int sumOD(long int A, long int B);
```

```
int main(void)  
{  
    long int T, A, B, i;  
    scanf("%ld", &T);  
    for(i=0; i<T; i++)  
    {  
        scanf("%ld %ld", &A, &B);  
        printf("%ld\n", sumOD(A, B));  
    }  
    return 0;  
}
```

```
long int sumOD(long int A, long int B)  
{  
    long int sA, sB, i, di, ;  
    sA = (A/2)*(A/2);  
    sB = ((B+1)/2)*((B+1)/2);  
    for(i=2; i<=B; i+=2)  
    {  
        di=DivImp(i);  
        if(i<A)  
            sA+=di;  
        sB+=di;  
    }  
    return sB-sA;  
}
```

```
long int DivImp(long int n)  
{  
    if(n%2)  
        return n;  
    else  
        return DivImp(n/2);  
}
```

La función *main* tiene dos argumentos de entrada *argc* y *argv*.

```
int main(int argc, char *argv[])
```

El primer argumento (*argc*) es un entero que corresponde al numero de parámetros de entrada del programa cuando este es llamado. El valor mínimo es 1, dado que al menos tiene el llamado del programa.

El segundo argumento (**argv[]*) corresponde a un arreglo de cadenas para cada uno de los parámetros de entrada. El primer valor en el arreglo corresponde al nombre del programa.

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    int i;  
    printf("Numero de parametros: %d\n", argc);  
    for(i=0; i<argc; i++)  
        printf("%s\n", argv[i]);  
    return 0;  
}
```

```
>> ./main  
Numero de parametros: 1  
./main  
>> ./main 1.2 4 6  
Numero de parametros: 4  
./main  
1.2  
4  
6  
>> ./main 1.2 4 Hola  
Numero de parametros: 4  
./main  
1.2  
4  
Hola
```

Cada parámetros que es ingresado al main esta separado por el carácter nulo, el caractere nulo tiene un valor de cero (0). Si queremos contar el numero de caracatres para cada paranetro necesitamos localizar el valor nulo para cada elemento en el arreglo.

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    int i, j, val;  
    printf("Numero de parametros: %d\n", argc);  
    for(i=0; i<argc; i++)  
    {  
        j=0;  
        while((*argv[i]+j)!=0)  
            j++;  
        printf("%s\n", argv[i]);  
        printf("Numero caracteres: %d\n", j);  
    }  
    return 0;  
}
```

```
>> ./main  
Numero de parametros: 1  
./main  
Numero caracteres: 6  
>> ./main 1.2 12 Hola  
Numero de parametros: 4  
./main  
Numero caracteres: 6  
1.2  
Numero caracteres: 3  
12  
Numero caracteres: 2  
Hola  
Numero caracteres: 4  
>> ./main Univeridad de Guanajuato  
Numero de parametros: 4  
./main  
Numero caracteres: 6  
Univeridad  
Numero caracteres: 10  
de  
Numero caracteres: 2  
Guanajuato  
Numero caracteres: 10
```

```
./main 23 592
```

```
argc = 3  
argv[0] = "./main"  
argv[1] = "23"  
argv[2] = "592"
```

```
A = cad2int("23");
```

```
str[0] = "2"  
str[1] = "3"  
str[2] = 0
```

x=0;	j=0;	x*=10 = 0	str[0]-"0"=2	x+=(str[0]-"0") = 2
x=2;	j=1;	x*=10 =20	str[1]-"0"=3	x+=(str[0]-"0") = 23

```
B = cad2int("592");
```

```
str[0] = "5"  
str[1] = "9"  
str[2] = "2"  
str[3] = 0
```

x=0;	j=0;	x*=10 = 0	str[0]-"0"=5	x+=(str[0]-"0") = 5
x=5;	j=1;	x*=10 =50	str[1]-"0"=9	x+=(str[0]-"0") = 59
x=59;	j=2;	x*=10 =590	str[2]-"0"=2	x+=(str[0]-"0") = 592

UNIVERSIDAD DE GUANAJUATO

